

university of groningen / faculty of science and engineering / computing science

Feature selection performance assessment

June 2020

MSc Research Internship INMSTAG-08

Student: J.G.S. Overschie

First supervisor: dr. G. Azzopardi

Second assessor: A.M.J.A. Alsahaf

Abstract

The evaluation of feature selection algorithms is a many-faceted problem, which has been conducted in different ways in the literature. With a unified theoretical framework lacking, authors resort to widely used methods of evaluation - which are not necessarily optimal. In this paper, all relevant aspects of the evaluation process are thoroughly analyzed, after which a set of sensible metrics is distilled. Using *a priori* information about relevant features, promising metrics to measure feature subset quality include the ROCAUC- and mAP scores. Furthermore, measures of stability and statistical integrity testing are given. An implementation of the pipeline, fseval, is made publicly available. Finally, a quantitative experiment implements and discusses the newly proposed set of metrics, which shows the new metrics to be of value and able to foretell feature subset prediction performance.

1 Introduction

Feature selection has since long played an important role in data mining and prediction tasks, where the removal of irrelevant or redundant features can decrease the required storage space of a dataset without decreasing prediction performance. Even, in some cases prediction performance can increase, due to less noise in the input signal. In the current age and time, however, feature selection is becoming ever more relevant - there now exist cases where it becomes unfeasible to store and process all features of a dataset in its entirety; even though computational power and storage space increase, the improvements in hardware performance are now outpaced by the ever-larger datasets available [35]. Hence, there is a need for reducing dataset dimensions.

Datasets have grown in both dimensions and amount of samples over the last couple of decades (UCI repository growth [2]). Internet traffic increased, new and more accurate measuring devices came to market, and disk storage space became cheaper. Whilst in most areas both the amount of dimensions (p) and the amount of samples (n) increased, in some fields the amount of samples remains sparse relative to increasing dimensionality; bio-informatics, for example. Whilst the process of obtaining samples can still be intrusive, sensory devices got more accurate, causing more information to be available on a single sample. These datasets are of 'large p - small n' shape ($p \gg n$), which are challenging to reduce dimensions for due to the small amounts of samples. These, and other dataset shapes can benefit from applying feature selection techniques.

Existing literature on feature selection is extensive. Many methods exist, taking various approaches. What remains a common point of concern, however, is a consistent and reliable way to evaluate feature selection method performance. This need is expressed in several papers, e.g. 'a unifying theoretical framework is lacking' [15]. Most often, feature selection is applied to real-world datasets, after which the performance of a prediction task is assessed using labelled data. In this way, different feature selection methods would output different feature subsets which in turn would influence the validation predictor performance. The problem with this approach is that the evaluation becomes reliant on the choice of the validation predictor, as well as the fact that the 'correctness' of the chosen features is not taken into account. Commonly missing in papers is also an analysis on algorithm stability and a statistically sound argumentation for superiority. Therefore, a more controlled and more systematic way of arguing one method's performance over another is desired.

The goal of this paper is to provide an analysis on common ways of evaluating feature selection method performance, by discussing recent literature. With a better understanding of reasonable ways to argue for any given method, a distillation is then made to recommend a practical evaluation pipeline for feature selection methods.

This research contributes in three ways. First, ways of effective feature selection performance assessment are explored, in which different ways of assessing performance in literature are compared and a reasoning is provided as to what metrics are sensible to use in various cases. Second, a quantitative research using the proposed set of metrics is performed, comparing various promising supervised feature selection methods for classification problems. In the empirical part of this research, **scope**

was reduced to focusing on supervised feature selection for classification problems using datasets that exhibit 'conventional flat features' (terminology available in Chapter 2). However, the proposed feature selection evaluation pipeline should also be applicable to evaluating unsupervised setups too. Last, the proposed pipeline was implemented in Python and published as a package on Github, allowing authors to easily implement the pipeline themselves.

The paper is built up in several theoretical chapters, followed by quantitative analysis results, a discussion and a conclusion. Chapter 2 explains the basic feature selection process and lays out basic terminology. Chapter 3 discusses how to best evaluate feature selection performance, by first discussing the state of affairs and then proposing a new set of metrics to use in evaluation. Chapter 4 walks through a quantitative experiment using the newly proposed set of metrics. Finally, Chapter 5 discusses directions for future work and concludes the research.

2 Reducing dimensions

Among the subject of reducing dataset dimensions, there exist a common terminology that is used among the literature. Whilst some terminology is synonymous, other seemingly related terms mean different concepts entirely.

Feature selection and **feature projection** are both terms relating to the concept of dimensionality reduction [8], however, there exists an important distinction between them. Whilst in the process of feature selection, relevant dimensions are sought and selected without altering their input values, in the process of feature projection (also called *feature extraction*) data transformations are applied, mapping the original data onto a lower-dimensional space. Although the two the methods are different, the two aim to achieving the same goal, and are thus encountered in similar contexts. In this paper, the term *feature selection* is used as a term to indicate the general process of obtaining a feature subset with reduced size without transforming the data - note that any feature selection method might transform the data in the algorithm however it likes internally - the stated terminology is only concerned with the eventual output of the feature selection algorithm.

Common methods of feature projection are *Principle Component Analysis* (PCA) for the supervised case and *Linear Discriminant Analysis* (LDA) for the unsupervised case. Both PCA and LDA take a statistical approach to detecting feature interactions, which not always results in an optimal feature set for prediction. Rather, machine learning techniques can be used to select a more optimal subset.

2.1 Common approaches

Methods can be roughly subdivided between filter-, wrapper- and embedded methods [7], which lays out a common taxonomy within the field.

Filter methods use some scoring mechanism to compute 'usefulness' for each feature, without applying a learning algorithm. Having applied some ranking criterion, often a feature ranking is produced, after which some thresholding operation can be applied to select features. Although filter methods are often computationally light and do not overfit due to the absence of a learning algorithm, filter methods might miss out on more complex feature interactions, causing a non-optimal subset as a result. Also, choosing a suitable threshold to use can be difficult.

Wrapper methods, on the other hand, use some learning algorithm to determine a suitable subset of features. A search is conducted over the space of possible feature subsets, eventually selecting the subset that has the highest validation score in the test set using a chosen learner as a predictor. Characteristics that define wrapper methods lend themselves similar characteristics to traditional optimisation problems; although an exhaustive search might yield an optimal solution, such a solution might not always be feasible due to its great time complexity. For this reason, in some applications a liberal filter is applied first, before running a wrapper method.

Embedded methods seek to combine the training task and feature selection. Given some suitable learner, features are weighted during the training process, producing either a feature ranking or a feature subset afterwards. e.g. some learners compute feature importance scores as part of

their training process, which can then be used in combination with some threshold to select relevant features. Having already trained the model, subsequent prediction tasks can benefit from increased prediction speed by using less data.

Hybrid methods is any method that is not classifiable by a single category, but rather lends from multiple categories. Hybrid methods can, for example, combine filter and wrapper methods [17], by first applying a computationally efficient filter and refining the result by using a wrapper method. Another paper [9] describes its approach as hybrid due to both adding- and removing features in the feature selection process - exhibiting both forward- and backward selection at the same time. Lately research was also put into examining *Ensemble* feature selection methods [3], which combines outputs of multiple selectors and decides useful features accordingly using some voting committee. Ensemble methods can be seen as hybrids or are seen as a category on its own.

2.2 Types of features

An important consideration in choosing a suitable feature selection for any task, is what structure the concerning data has, if any at all. Data might exhibit tree, graph, or grouped structures, which is essential information when detecting feature interactions and determining useful features. Support for structured data is relatively new in the field, and has not been an extensive point of concern for many feature selection algorithms in the past. Many traditional feature selection algorithms focused primarily '*conventional flat*' features [24], in which the assumption is made that the data is independent and identically distributed (*i.i.d.*). This assumption is widespread among many machine learning algorithms, though, since in many applications datasets are normalized to fit the i.i.d. condition before they are used.

Conventional data are opposed to more complex data structures, i.e. datasets with 'structured features', as coined by the proposed taxonomy from J. Li et al (2017) [24], but also to linked data and *streaming data*. In streaming data, the quality of an initially selected feature subset can be improved as more data comes in, and a feature selection algorithm is able to benefit from a larger distribution of samples. Adapting existing feature selection algorithms to fit the demands of streaming data proved to be a non-trivial problem. Nowadays many companies and institutions have to deal with data volumes that easily exceed the boundaries of in-memory storage capacity - limiting the data scientist to train on only subsets of the entire datasets. Smart sampling is therefore needed to retain a representative sample distribution.

The scope of this research is limited to only the most common type of features, **conventional-, flat features** (*i.i.d.*).

3 Evaluating performance

Evaluating the performance of feature selection algorithms is a non-trivial problem. Among a wide range of metrics, no consensus exists among researchers, leaving many papers to present outcomes in different ways [15]. Even though in absence of a single consistent evaluation pipeline across the field, many scholars adhere to methods that are 'widely used' [32] [24], there might be methods of evaluation that are more meaningful.

3.1 State of affairs

Recommendations for metrics have been given in previous papers, most often when discussing future work. Arguments are made for relevant aspects to evaluate, such as in Chandrashekar (2013) [7]:

> *"a feature selection algorithm can be selected based on the following considerations: simplicity, stability, number of reduced features, classification accuracy, storage and computational requirements"*

Of these aspects most proposals focus mainly on number of reduced features, classification accuracy and computational requirements. Let us explore what classifiers and corresponding metrics are used

in papers across the field. Afterwards, evaluation aspects are covered that are not present in the aforementioned set.

Supervised feature selection is most commonly evaluated by running some predictor over the concerning dataset using the selected feature subset, obtaining the easily interpretable accuracy metric in the classification case. Predictors often used in the literature include k-NN [1] [26], SVM's [7], Decision Trees [24] and Naïve Bayes [21]. Metrics used are often classification accuracy or in some cases average error rate [19], validated using some n-fold cross validation, commonly 5- or 10-fold.

Unsupervised feature selection is often evaluated using a k-means predictor, as noted by Solorio-Fernández (2020) [32], in which 68 papers were reviewed. Metrics include Clustering Accuracy (ACC) and Normalized Mutual Information (NMI).

Stability, on the other hand, is not widely used in theoretical or quantitative argumentation. Even though stability was recommended as a relevant evaluation metric by Chandrashekar (2013), not many papers explicitly argue for the stability of their method; the metric is called an *"overlooked problem"* in Chandrashekar (2013). In many papers this metric is still regarded as a future work for solidifying any experimental results - the development of algorithms that achieve both high classification accuracy and high stability is still seen as a 'challenging' problem by Tang et al (2015) [34].

The trend seems to be turning though, with more authors becoming aware of the importance of stability. Our reliance on machine learning is ever-increasing, and so does the demand for interpretability and reliability of the algorithms. Take for example a bio-medical application, in which feature selection is used to select genes in a gene sequencing analysis. Any expert in this domain field will feel more confident if an algorithm produces stable results given a varying sample population. This is how we define stability: given small changes in the sample data, the output of a feature selection process ought to be stable, i.e. the sensitivity of the selector to data perturbations. This metric for measuring an ability to generalize to achieving consistency has been long taken into account into prediction tasks, but not in feature selection.

Scalability is another point of attention that only recently caught more attention. The extra demand of algorithms to allow for parallel execution has been imminent as data grew tremendously large. Even, multi-core processing can lack in terms of performance, hence introducing the need for algorithms that can run in distributed fashion. Distributing a dataset over multiple machines poses challenges to some existing methods, though. Some current methods require the 'full dimensionality' of the dataset to be available in-memory [34]. Yet, other methods require each sample to be visited multiple times, e.g. to apply a sample re-weighting strategy in order to converge. For these reasons, dividing any dataset workload onto multiple workers is a non-trivial problem; no generalized solution exists for cutting the dataset into chunks.

It is up to individual algorithms to find suitable ways of supporting parallel solutions and more importantly, support cases where data is too large to fit in-memory, i.e. apply distributed computing. Recent strategies overcome the issue of working with less samples by retaining only those samples that are most representative of the data - eliminating the need for working with a full sample population. Although few in numbers, there exist proposals for distributed dimensionality reduction methods [25], using 'divide-and-conquer' techniques. Aggregating disjoint results would make for a performance similar to that of a centralized solution.

3.2 Meaningful evaluation

Let us revisit and complement all previous theory, and extract a sensible evaluation pipeline to use. First, each evaluation aspect is treated separately, followed by a recommendation, then an overview is drawn of the entire proposed evaluation pipeline.

3.2.1 Simulations

Many proposals test their validity on real datasets. Although testing on real datasets might feel more intuitive since it is also the eventual domain of application, they may not prove the best fit for feature

selection method evaluation. With any dataset for which we cannot exactly control its properties, we are resorted to a smaller available set of validation measures, such as classification accuracy. However, introducing synthetically generated probe variables, or even generating all benchmark data synthetically can provide more reliable results in a controlled environment.

Partially synthetic datasets are spoken of in literature since long, using datasets that are real but altered by injecting more data. In [15] the authors speak of 'probe variables', which are used to discard any variable that scores lower than any of the probes. If the probe variables are set to be random variables, a simple way is obtained to apply a threshold to cutting off features from the selected feature subset, i.e. by introducing known noise into the dataset we can construct more thoughtful cut-off thresholds.

Completely synthetic datasets, on the other hand, can allow for more sophisticated metrics to be used. Possibilities for new evaluation metrics are for example described in [32]: *"Evaluation in terms of the redundancy of the selected features"* and *"Evaluation in terms of the correctness of the selected features"*, the latter of which requires us to know what features are informative *a priori* - which is accomplished with synthetic generation. Controlling all facets relevant to the quantitative analysis manually makes for a *Simulation study*, which is argued for in [37] as follows:

> *"Simulation studies such as these facilitate proper evaluation and comparison of methodologies because a simulation study can be designed by systematically varying key experimental conditions, and the ground truth of the dataset is known i.e. we know which features are relevant vs. irrelevant, we know the pattern of association between relevant features and endpoint, and we know how much signal is in the dataset."*

Indeed there seems to be a trend toward using synthetically generated datasets. In a review paper [4] the authors argue that synthetically generated datasets can yield statistically sound results because of the fact no inherent noise or redundancy will obstruct the experiment process.

A priori knowledge of relevant features allows us to evaluate feature selection results in a more meaningful manner. Algorithm performance measurements can take advantage of using classification metrics familiar to many - see Section 3.2.2 - which allows us to better understand any algorithm's selection process in comparison to doing unsupervised evaluation. Another benefit is gained by relying on a priori knowledge in the validation process, which is that the evaluation process is no longer dependent on the choice of the validation classifier. Removing the validation classifier from the evaluation process completely can make for a more universal way of evaluating feature selection methods: whilst previously reported feature selection method scorings were dependent on the validation classifier and would per definition not be comparable across papers, using a priori based metrics allows for better comparability across papers.

In other, more recent papers simulation studies are conducted as well [6] [33] [25], concluded by a small section depicting a 'real data analysis' to conclude the point. For these reasons, we recommend use of simulation studies to evaluate feature selection methods.

3.2.2 Feature subset quality

Based on the new assumption of knowing the groundtruth in the evaluation process, possibilities are opened for use of many common classification metrics to evaluate feature subset quality. One can now compute the ratio of 'amount of selected relevant features versus the total amount of relevant features present in the dataset' and the ratio of 'selected relevant features versus the total amount of selected features'; recall and precision respectively. The amount of 'true negatives' will be the amount of features that were not selected and are not informative - hence implicitly correctly identified as irrelevant. With recall and precision now available, also the f-scores can be computed, as well as the true-positive-rates, false-positive-rates and therefore the ROC curve and AUC-scores.

Statistical metrics used for machine learning evaluation can be formulated in such a way to be suitable in the context of a feature selection evaluation process. Given a feature subset computed by a feature selector, metrics can be defined as such:

Negatives are indicated as the remaining dataset features not in the feature subset, i.e. $Negatives = p - Positives$ where p is amount of dataset features. Using the above definitions more sophisticated

		Amount of
True Positives	(TP):	selected 'informative', i.e. groundtruth features
False Positives	(FP):	selected 'uninformative', i.e. redundant/irrelevant features
True Negatives	(TN):	unselected uninformative features
False Negatives	(FN):	unselected informative features

metrics can be defined:

$Recall$	$= \frac{TP}{TP+FN}$	or $True\ Positive\ Rate$
$Precision$	$= \frac{TP}{TP+FP}$	
$False\ Positive\ Rate$	$= \frac{FP}{FP+TN}$	

ROC- and PR curves can now be computed, using the above definitions. However, one must take into account the fact that any dataset is represented by multiple replicas - and insight on aggregated results are desirable. Therefore, a coherent way of computing the mean curve must be found. For the ROC curve, the process is straightforward - a mean curve can be computed by first interpolating each curve over a set of equally spaced false-positive-rate (fpr) interval steps, e.g. $\{fpr \mid fpr \in [0, 1]\}$ with 100 intervals, and then averaging the interpolated recall values. Given the aggregated ROC curve, the ROC-AUC metric can easily be computed. For the PR curves, however, the solution is less trivial - due to the non-monotonic nature of the curve 'wiggles' are commonplace. Applying linear interpolation is known to cause an overly-optimistic performance estimate [10]. Therefore, a more sophisticated way of interpolation is required for PR curves.

Interpolating a PR curve was written about in [31] and its proposed measure was described in [13]:

$$p_{\text{interp}}(r) = \max_{\tilde{r}:\tilde{r}\geq r} p(\tilde{r}) \tag{1}$$

The formula defines the interpolated precision, $p_{\text{interp}}(r)$, for any recall value as the maximum precision value of all observations at the first recall level to the right with non-null observations. Such 'non-null' precision observations are defined as $p(\tilde{r})$ with its recall level $\tilde{r}$. By means of this interpolation method, wiggles are overcome, yielding a more consistent curve.

AUCPR, or Area Under the Precision/Recall Curve is a useful metric to summarize PR curve performance. There exist multiple ways of quantifying the metric, however, as was shown in [5], with methods yielding different levels of robustness in the estimation process. In the paper a range of methods are discussed, e.g. a Trapezoidal Estimator and Average Precision. The authors, Boyd et al, empirically validated method robustness and, among others, concluded *Average Precision* (AP) to be effective. Since the metric is often used in literature and is robust, it is a sensible metric to express the AUCPR. It can be defined as [38]:

$$\text{AP} = \int_0^1 p_{\text{interp}}(r)dr \tag{2}$$

Which is an integral over the recall range of $[0, 1]$ to obtain the area under the precision-recall curve. A finite implementation is constructed by summing the areas under each separate precision plateau. The amount of plateaus is finite and are at the maximum the amount of possible recall values, which in turn are the amount of informative features. So, rather than having to approximate the area by computing areas at equally spaced intervals, in this fashion, we obtain an accurate estimate for the Average Precision. The finite average precision is defined as:

$$\text{AP} = \sum_{k=1}^{n} P_{\text{interp}}(k)\Delta r(k) \tag{3}$$

Where n is the amount of precision plateaus and k the current plateau, for which an interpolated precision is available as $P(k)$ and $\Delta r(k)$ the change in recall given the precision plateau.

Aggregated PR curves can now be obtained by computing the mean curve from all interpolated curves - similar to how the ROC curve was aggregated. Now, however, we do not require interpolation in the aggregation process since the PR curves were already interpolated at fixed recall points. To aggregate the AP scores, one can take the mean of all replica AP scores, more commonly referred to as the mAP score:

$$\text{mAP} = \frac{1}{m} \sum_{i=1}^{m} \text{AP}_i \tag{4}$$

Here, m denotes the amount of replicas ran for a specific dataset, with AP_i be the average precision for the ith replica.

Using these metrics, a useful set of statistical tools are laid out for evaluating feature subset quality, and thereby also feature selection method performance. Rather than just using validation estimators to measure feature subset prediction power, by comparing against a groundtruth the feature selection method evaluation process itself became supervised - possibly providing the process with more meaningful feedback.

3.2.3 Stability

Stability is an integral metric of any algorithm, for in many applications, only robust algorithms can be systematically relied on. A measure for quantifying stability that comes easily to mind might be the Hamming distance between multiple pairs of algorithm runs, i.e. ran on the same dataset with varied sample populations. Indeed, in [12] a measure based on Hamming distance is proposed. Reports also exist on numerous other approaches, like an entropy based measure [22], a measure based on the cardinality of intersection measure [23] and a measure based on correlation coefficients [18].

Desired properties of any measure are important to define in concrete manner. To compare the strength of any of these measures, the general objective for exactly what information we want to convey in a stability metric should be verbalized first. In [27], three desired stability metric properties are expressed: (1) *Monotonicity*, (2) *Limits* and (3) *Correction for chance*. Given the proposed measures in the literature, only few measures satisfied all properties. In a more recent paper [29], however, the authors extend the set of desired properties to include another two: (4) the stability estimator must be *Fully defined* and (5) *Maximum Stability* $\leftrightarrow$ *Deterministic Selection* - meaning that a maximum stability value should be achieved if-and-only-if all feature sets are exactly identical.

A stability measure was proposed in [29], using the set of newly proposed desired properties. Having explored a statistically sound method to define a metric satisfying all five properties, the authors proposed a novel stability estimator:

$$\hat{\Phi}(\mathcal{Z}) = 1 - \frac{\frac{1}{d}\sum_{f=1}^{d} s_f^2}{\mathbb{E}\left[\frac{1}{d}\sum_{f=1}^{d} s_f^2 | H_0\right]} = 1 - \frac{\frac{1}{d}\sum_{f=1}^{d} s_f^2}{\frac{k}{d}\left(1 - \frac{\bar{k}}{d}\right)} \tag{5}$$

Here, $\hat{\Phi}$ resembles the stability estimate. It takes as an input $\mathcal{Z}$, which resembles a collection of feature sets chosen by the designated feature selection algorithm. We define $\mathcal{Z}$ as $\mathcal{Z} = \{s_1, ..., s_M\}$, where s_i is one feature subset of size d features, with d the amount of dimensions of the dataset and M the amount of bootstrap samples - i.e. the amount of replicate datasets with equal probability distribution and dataset properties used to validate stability. Furthermore, the authors define s_f^2 as the unbiased sample variance of the selection of the fth feature and $\bar{k}$ as the average number of features selected over the M feature sets. Given this paper's goal, there is no necessity for going into further mathematical details - for this we refer to the paper itself.

What is important, is that this new measure satisfies all desirable properties for quantifying stability, as was proven in the paper. Accompanying the novel definition, are instructions for computing confidence intervals and for performing a hypothesis testing for comparing various method stabilities. Exploring feature selection stability values given a set of parameter choices not only

allows for choosing better hyperparameters, but also allows for comparing stabilities over various feature selection methods. Therefore, we recommend use of the metric defined in (Eq 5) to quantify feature selection method stability.

3.2.4 Algorithm complexity

Another metric to be taken into account is algorithm complexity, which manifests itself in three interlinked aspects: time-, storage- and algorithm simplicity. Like the classical principle *Occam's razor* implies - there at all times exist a preference for simpler models over more complex ones, especially in the case both accomplish the same feat. So, an understandable model is preferred that performs limited computational steps in order to restrain time- and storage complexity from rising too high.

Although measuring algorithm time- and storage requirements does provide some understanding, more insightful would be a theoretical description of the complexities in Big O notation. Theoretical complexities are harder to obtain though - leaving many authors to resort solely to measurements. Nonetheless, complexity analysis is recommended to be part of any feature selection method comparison, for it is a critical aspect to consider.

3.2.5 Statistical integrity

Given the above discussed scalar metrics (mAP, ROC-AUC, Stability) for multiple feature selection methods computed for multiple datasets, a statistical test must be applied to provide convincing evidence for any algorithm's superiority. Like explained in [11], many papers make implicit hypotheses acclaiming improved performance over existing methods. Although chosen metrics might have been appropriate to statistically show performance gains as significant, results are less reliable when left unvalidated by a statistical test. Therefore, a statistical verification step is required in the feature selection evaluation process.

Comparing multiple feature selectors is a non-trivial problem, which can be seen as the problem of comparing multiple *classifiers*. To compare multiple classifiers, [11] recommends the Wilcoxon signed ranks test for comparison of two classifiers and the Friedman test for comparison of more classifiers given multiple datasets. Accompanying the Friedman test, it is recommended to perform corresponding post-hoc tests, such as the Nemenyi test [28]. The usually popular ANOVA test was advised against because, given the context of machine learning, ANOVA assumptions are violated, e.g. ANOVA assumes samples are drawn from normal distributions and the requirement of random variables having equal variance.

A **Wilcoxon signed ranks test** and the **Friedman test**, are for these reasons recommended, for two- or more methods respectively, to statistically verify significant differences in performance of feature selection algorithms over the other using some summarizing scalar per feature selection method per dataset. The Friedman test is recommended to be accompanied with a post-hoc test, like Nemeyi's post-hoc test.

3.2.6 Pipeline

All facets of meaningful evaluation were discussed and several recommendations were made. Wrapping up, a recommendation for an entire evaluation pipeline is given.

Recommendations for meaningful feature selection evaluation are as follows. (1) First of all, it is recommended to use synthetic datasets, such that informative features are known *a priori* and datasets can be carefully designed to answer specific research questions, such as algorithm scalability or their ability to detect complex feature interactions. (2) Secondly, using now available groundtruth information, evaluation is recommended to benefit from analyzing common machine learning metrics, like the ROC- and PR curve, as well as summarizing metrics like the AUC- and mAP scores. (3) Third, stability should be measured using the tailor-made proposed measure from [29]. Note that computing algorithm stability does not require the groundtruth. (4) Fourth, algorithm complexity should be assessed in three different perspectives: simplicity, time complexity, storage requirements. A theoretical analysis is preferred. (5) Last, conclusions can be drawn by summarizing all computed

metrics in a statistical verification test, using either the Wilcoxon signed ranks test or the Friedman test accompanied with the Nemenyi post-hoc test. Having statistically verified the results, a more scientifically reliable conclusion can be drawn.

An implementation of the proposed evaluation pipeline was built as part of the research. The framework implements many of the steps required in perform meaningful evaluation and is publicly available on Github [1].

4 Experiment results and discussion

Using the newly defined set of metrics for evaluating feature selection methods, an quantitative study was conducted. In compliance with the idea of doing a simulation study, synthetic datasets were used. In this study, 76 synthetically generated datasets from [37] are used, for which each 24 *replicas* are considered - duplicate datasets but with different sample populations. The synthetic datasets cover various factors of influence to the feature selection task, like missing data, noise levels, sample sizes, dataset dimensionality and the types of feature interactions. All datasets possess a ground truth accordingly, which is resembled by the informative features an algorithm should select.

Testing was conducted on a selection of 4 supervised feature selection methods, using a mixed bag of methods that produce either a *feature ranking* or a *feature subset*. Other notable differences methods that exhibit *stopping criteria* versus ones that do not, as well as forward- versus backward selection. The lineup is described as follows:

- **ExtraTrees** [14]: ExtraTrees is a generic classifier used in a wide spectrum of applications, which works by creating an ensemble of (extremely) randomized trees. To use ExtraTrees as a feature selector, the feature importance scores are used to generate a ranking. This ranking is then reduced to a feature subset by cutting off features below the mean feature importance score, i.e. a threshold value defined as $\alpha = \frac{1}{p}\sum_{f=1}^{p} w_f$ and its feature subset as $\{f \mid w_f \geq \alpha\}$, where p is the amount of features, w_f is the fth feature importance score.

  Configuration for ExtraTrees was set to use 50 estimators, all other settings were otherwise set at the scikit-learn [30] default. Since the feature importance scores are thresholded at the mean, the method produces not a ranking, but a **feature subset**.

- **FeatBoost** [undisclosed]: FeatBoost is a greedy forward feature selection algorithm proposed by Alsahaf et al in 2020, based on a sample re-weighting strategy. FeatBoost requires one to choose two estimators as hyperparameters: one estimator is used for the internal boosting process, and should expose feature importance scores, whilst another is used to validate some selected feature subset.

  Configuration was set such to explore 3 different hyperparameter settings: (1) First, a baseline configuration, referred to as simply '*FeatBoost*'. FeatBoost using 'max features' set to 20 and the 'siso ranking size' set to 5 if amount of features $p \leq 100$ or set to 10 if amount of features $p > 100$. Furthermore, two XGBoost classifiers are used with 200 estimators and a max depth of 3. (2) Second, '*FeatBoost w/1-NN*'. Same as the former, but with k-NN ($k = 1$) as a validation classifier. (3) Third, '*FeatBoost Deep*'. Same as first, but both XGBoost classifiers having an increased maximum depth of 6. FeatBoost produces a **feature subset**.

- **MultiSURF** [37] [36]: MultiSURF is related to the original Relief algorithm [20] and was first proposed in a benchmark paper for Relief-Based algorithms by Urbanowicz et al, in 2018. In the benchmark paper, amongst a selection of Relief-Based algorithms in a comprehensive, MultiSURF was recommended as an overall good performer.

  Configuration for MultiSURF was set such to rank the entire set of features, otherwise all other settings lay at default. Subsequently, the MultiSURF 'top features' attribute was used to produce a **feature ranking**.

[1] https://github.com/dunnkers/fseval

- **Recursive Feature Elimination (RFE)** [16]: In RFE, features with low feature importance scores are pruned away in step-wise manner, starting with the set of all features, i.e. using backward selection. The algorithm must be configured using an estimator exposing its feature importance scores, requesting new importance scores every step, iterating until a desired amount of features is selected.

  Configuration: RFE was configured using a XGBoost classifier configured exactly like in FeatBoost's configuration (1) and was set to eliminate features until the subset size is of length 1. Also, RFE was configured such, that in each 'elimination' round, 2 features are eliminated. Then, RFE's exposed ranking attribute is used to produce a **feature ranking**.

Analysis is split up in various sections. First, an empirical perspective will provide insights on feature selection method performance on various dataset themes, given the previously discussed metrics. Finally, after all dataset themes have been discussed, statistical hypothesis testing will be applied.

4.1 Empirical observations

Although testing was conducted on many datasets, it is sensible to first explore effectiveness of various metrics given a tangible example. The '*3-way Pure Epistasis*' dataset was benchmarked at only 1 configuration, and hence lends itself well to a more thorough analysis. This first analysis is walked through in detail - once we established more trust in the chosen metrics, other datasets with more configurations can be analyzed en masse.

4.1.1 3-way Pure Epistasis

The 3-way Epistasis dataset records genetic information regarding Epistasis, which is a phenomenon in evolutionary biology describing how the mutations of one gene are dependent on other genes being mutated or not. In this case, 3-way interactions are considered - which also means that there exist 3 predictive features in the dataset. Another dataset property is 'heritability', which is a genetics term that in this context can be seen as the 'signal magnitude' [37]. Heritability $h = 0$ resembles complete noise whilst a clean dataset is defined with $h = 1$, which in case the dataset is completely clean could theoretically lead to flawless predictions. For '*3-way Pure Epistasis*' the heritability value is $h = 0.4$. Furthermore, the dataset is of dimensionality $p = 20$ and $n = 1600$. Like all other datasets, the dataset contains 24 replicas all generated using the same properties but exhibiting a different sample population.

Feature Rank	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Feature Index	**19**	16	15	2	**17**	14	6	8	12	**18**	7	10	11	3
Informative?	**Y**	N	N	N	**Y**	N	N	N	N	**Y**	N	N	N	N

Table 1: Output for the ExtraTrees feature selector using a mean-importance score threshold, ran on the '3-way Pure Epistasis' dataset, replica 1.

Table 1 presents the **raw output** of 1 feature selection algorithm, ExtraTrees, given 1 replica. The algorithm constructed a subset with 14 features and managed to select all informative features, however, uninformative features were selected, too. Like argued in Section 3.2.2, there exist various ways to quantify performance. Let us explore several.

Figure 1 depicts **Precision and Recall** metrics, for ExtraTrees. The left-hand plot depicts 1 replica and uses the data in Table 1 and the right-hand plot depicts all replicas. Whilst recall rewards an algorithm for selecting informative features in monotonically increasing fashion, precision both rewards and penalises. Whenever after a series of unsuccessful iterations, a useful feature was selected, precision jumps up - causing a 'wiggle' in the chart as theoretically explained in Section 3.2.2. For this reason interpolation was applied as according to the proposed evaluation pipeline, as seen on the right plot. Also on the right plot are text annotations, indicating how

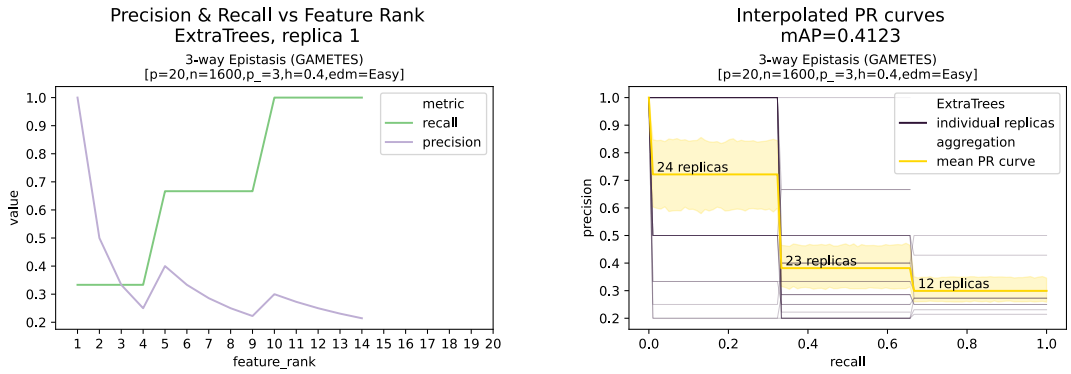


Figure 1: **Left**: Both precision and recall plotted against feature ranking. Metrics are raw unterinpolated data, for ExtraTrees, replica 1. **Right**: Interpolated Precision/Recall curve for ExtraTrees, all replicas. **Both** display the '3-way Pure Epistasis' dataset, for the ExtraTrees feature selector.

many replicas support the mean-PR curve at various points, i.e. not all replicas manage to select all informative features. In scenarios as such, where only several of the replicas manage to select all informative features, the mAP scores is still a representative summary of all replicas: the mAP score is computed as the mean of all *Average Precision* scores, not the Area Under the Curve - one way to think of it in this case is as a weighted AUC score.

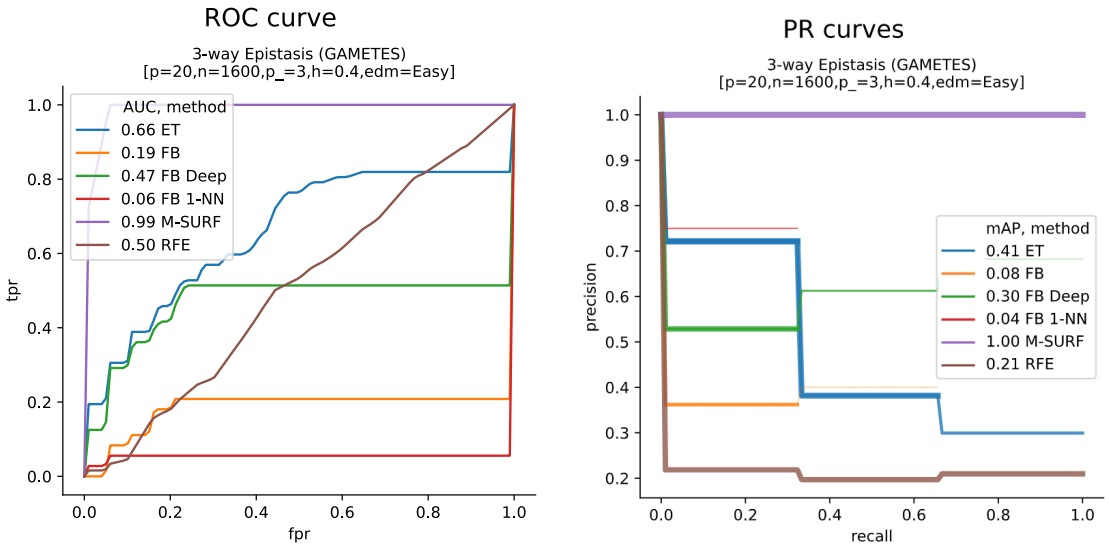


Figure 2: All ranking methods depicted in both the ROC- and (interpolated) PR space, for the '3-way Pure Epistasis' dataset. Accordingly to the ROC- and PR curves, respectively the ROC-AUC and mAP scores are attached.

Figure 2 depicts **ROC- and PR curves** for all ranking methods. Whilst in the PR space algorithms are penalised for selecting useless features, in the ROC space algorithms are only rewarded (monotonically increasing) for selecting the right features, as fast as possible: the false-positive-rate will increase when selecting uninformative features, with the true-positive-rate only increasing when selecting informative features. It can be observed that in this behavior, stopping criteria are not taken into account - which the PR curve does manage to do: whilst in the ROC space RFE is only moderately penalised for selecting many false positives before selecting true positives, RFE relatively scores considerably worse in the PR space. It should be noted, though, that the PR curves

might be overly optimistic due to the interpolation applied: all curves start by default at the (0 recall, 1 precision) point. This can clarify the perfect scoring for MultiSURF in the mAP score but not the ROC-AUC score. In the PR plot, lines of varied thickness are shown. Like explained in Figure 1, different PR 'plateaus' have different levels of replica support for their lines. The thicker the lines is, the more replicas support the line. This can help interpret, for example, the result of *FeatBoost 1-NN*, where very few replicas managed to get a reasonable score, resulting in a thin line, accompanied by a low mAP score.

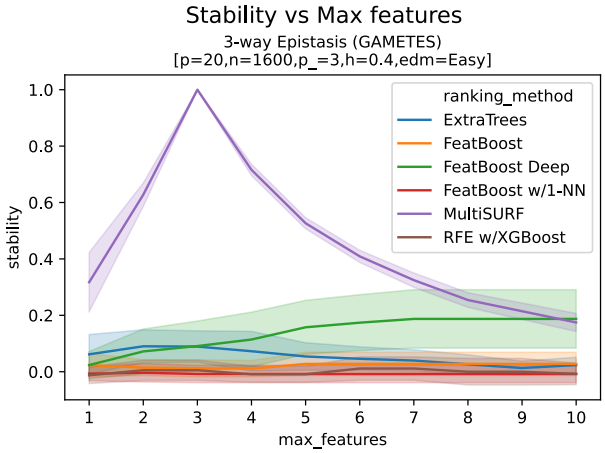


Figure 3: Stability versus Maximum feature subset size, depicted for all ranking methods for the '3-way Pure Epistasis' dataset.

Figure 3 depicts the **Stability** metric, using the measure as explained in Section 3.2.3. Since the metric requires feature subsets rather than feature rankings, feature rankings are cut off at various points of maximum subset sizes, k. In the figure, stabilities have been computed for cut off points $k \in [1, 10]$. It can be observed that only MultiSURF produced a reasonably stable subset, peaking out at 3 features - which is consistent with the amount of predictive features of 3. Raw data shows us that indeed MultiSURF produced a completely stable subset at a subset size of 3 - selecting all informative features in all replicas. Lower stability in smaller subset sizes are due to the algorithm selecting informative features in different *order*, i.e. there exist no ranking preference amongst the informative features themselves.

Figure 4 depicts **Classification Accuracy** scores for all ranking methods. The charts should not function as a general performance measure, like argued in Section 3.2, the charts are included, however, to convince the reader of the evaluation correctness of the previous steps: the previously shown metrics should be able to show what feature selection methods constructed powerful feature subsets and should hence perform well in any subsequent prediction task. Two predictors are considered: *k-NN* ($k = 1$) and *XGBoost*. For results of *Gaussian Naïve Bayes* and a (linear) *Support Vector Machine*, see Appendix A, Figure 12. All ran at default hyperparameter values.

Careful interpretation of the plot (Figure 4) is required: the process of aggregating the classification accuracy curve over various runs is open to graphical deceptions similar to what is seen in the Precision/Recall curves. Possible distortions are rooted in various algorithm runs stopping at different subset sizes due to stopping criteria, giving outliers possibility to distort the mean classification accuracy value. Whilst for the P/R curves it is possible to resort to the mAP metric, in a classification accuracy lineplot the supporting 'weight' of the mean curve is not directly visible. Therefore, in Figure 4, a 'line width' dimension was added, resembling the replica *support* the mean curve has. A thicker curve means more replicas supporting the curve. It can be observed that MultiSURF performs well across all replicas - all replicas were instructed to rank all the features. For FeatBoost Deep, however, there exist outlier results with very little support - which is where, if simply the mean were taken, the mean curve would have been distorted. Curve support is weighted heavily, though, since according to the mAP and ROC-AUC scores, ExtraTrees evaluates higher, although

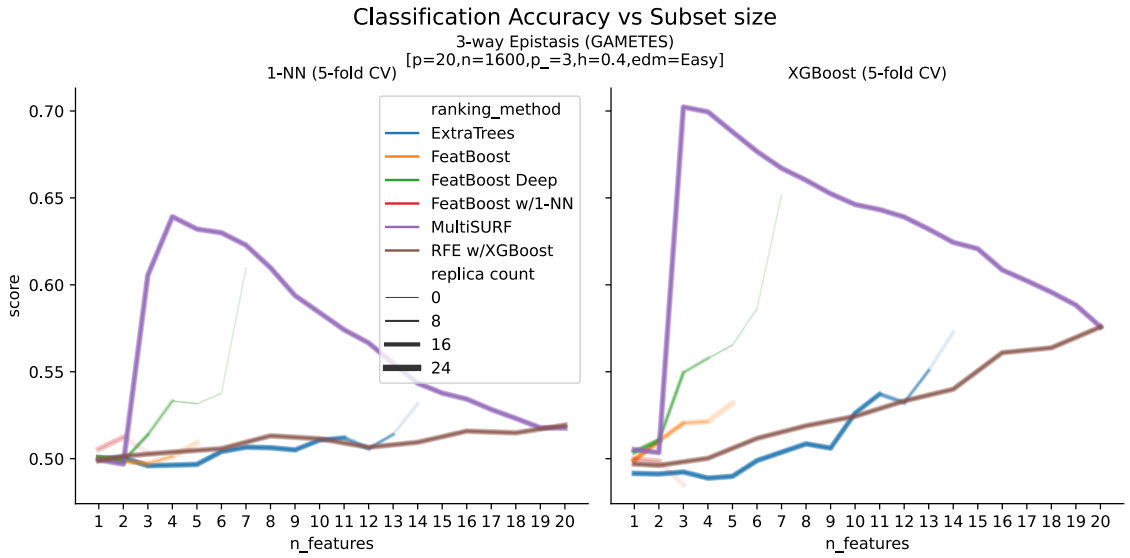


Figure 4: Classification Accuracy versus Feature Subset size, for the '3-way Pure Epistasis' dataset. Each replica produced 5 measurements with 5-fold CV. Results were averaged over 24 replicas, with the replicas partaking in the averaging process indicated by line width. Subsets were constructed by iteratively adding features of lower ranking scores for feature rankings, and similarly by adding features to a feature subset until its full size is reached.

in this validation stage it becomes clear that ExtraTrees does not perform so well - but it does so more consistently, i.e. with more curve support. According to what the earlier evaluation metrics were able to tell- the mAP and ROC-AUC scores represent feature selection method performance reasonably well.

To summarize all observations, mAP and ROC-AUC scores are able to foretell feature subset performance in a classification scenario. Much weight is put to the amount of replicas reaching their respective recall, precision or classification accuracy scores - which is in fact, sensible: an algorithm to perform well, but inconsistently, would have insufficient stability to be reliable and should thus score lower on feature selection evaluation metrics. In order to analyze other dataset themes *en masse*, a representative choice has to be made between the various metrics, initiating the need to answer an important question: which plot best resembles feature selection algorithm performance?

Because of the ambiguity of the replica curve support that exists in the PR curves, in the further evaluations of other datasets solely ROC curves will be used. Even though this ambiguity has been tried to made visible, for example by means of text labels in Figure 1 or by means of curve thickness in Figure 2, still the lines do not faithfully represent the actual (consistent) performance and might therefore be misleading. It must be noted that the ROC curve might be overly-optimistic w.r.t. algorithms that have no stopping criteria - this can, however, be compensated by including the mAP metric in a later analysis stage. The mAP scores do faithfully represent performance over all replicas and will later be used in the statistical integrity testing section. Note that another solution would be to normalize replica subset sizes to be of equal length, e.g. by cutting off at a certain threshold or fixed length, or, to instruct feature selection algorithms to select a specific number of features - see Future work, Section 5.2. That said, due to the reasons above, in the remainder of analysis ROC curves will be used.

4.1.2 Main Effects

In the '*1 Main Effect*' dataset theme there exist a single informative feature, that should be relatively easy to detect in comparison to feature interactions.

Figure 5 depicts ROC curves for the given dataset theme. It can be observed that all algorithms

perform decently well, except for *FeatBoost 1-NN*, which seems to miss the informative feature in some cases.

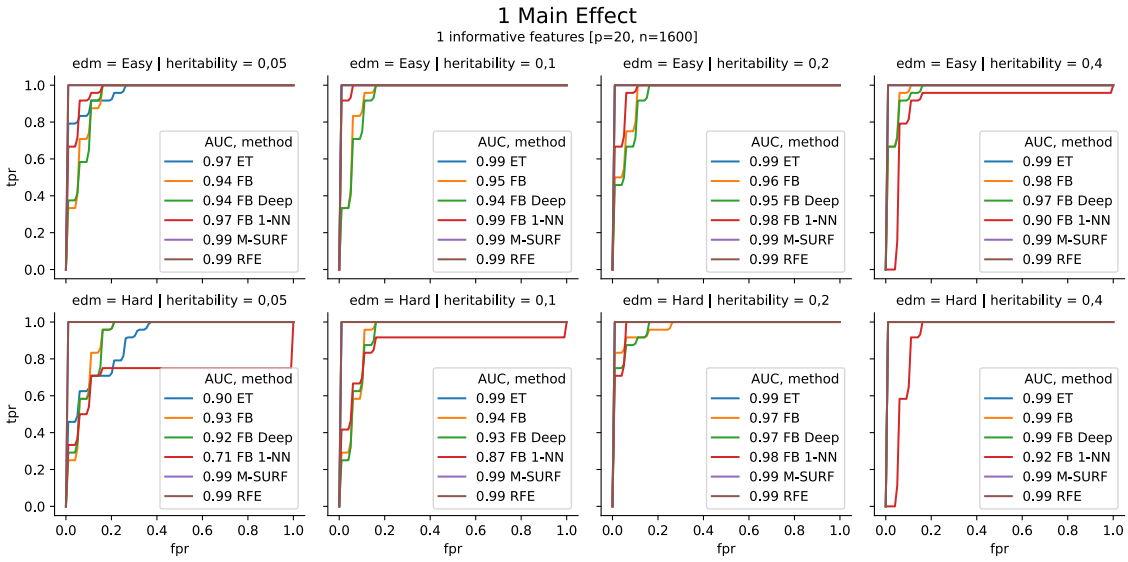


Figure 5: ROC curves for '*1 Main Effect*' dataset theme. 8 configurations were tested, with varying heritability and model difficulty levels (*edm* in plot).

4.1.3 2-way Epistasis

The '*2-way Epistasis*' dataset represent 2-way feature interactions and are hence more difficult than the a 1 main effect.

Figure 6 depicts ROC curves for 2-way epistasis. Immediately visible is the worsened performance in the 2-way interactions in comparison to 1 main effect. All algorithms seem to have difficulties with the hard models, although the FeatBoost algorithm configurations have most difficulty with high-noise (low heritability), hard models.

4.1.4 XOR benchmarks

The XOR models are 'clean' datasets, which means the datasets are generated without any noise using a heritability parameter of ($h = 1.0$). The models include from 2 up to 5 informative features, influencing the difficulty of detecting the interactions.

Figure 7 depicts ROC curves for XOR benchmark models. An interesting observation is that MultiSURF performance, which has previously been very steady, plummets quickly when the amount of feature interactions is increased. Although its AUC is still higher than the FeatBoost methods, it comes at a large false-positive-rate. ExtraTrees performance ranks highest for the high amount of feature interactions ($p_{=}5$), which might imply that ExtraTrees is better able to detect more complicated feature interactions.

4.1.5 Total number of Features

Lastly, a dataset theme varying the total amount of dataset features whilst keeping other variables constant. Up until now all other datasets had a relatively small amount of features, which allowed us to answer questions about the types of complexity different feature selection algorithms can handle, but not exactly about *scalability*.

Figure 8 depicts ROC curves for Total Number of Features datasets. For most feature selectors, an increased number of features has a tremendous impact on the quality of the chosen subsets.

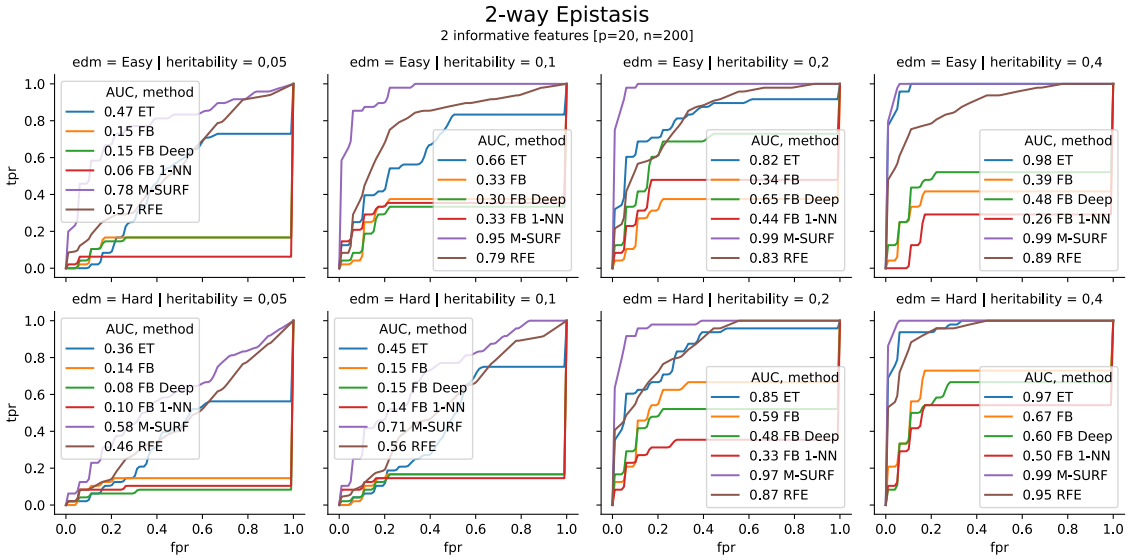


Figure 6: ROC curves for '*2-way Epistasis*' dataset theme. 8 configurations of varying heritability and model difficulty levels were tested, with 4 different configurations of amount of samples. $n = 200$, $n = 400$, $n = 800$, $n = 1600$. See Appendix A, Figure 14 for plots with $n = 400$ and $n = 1600$.

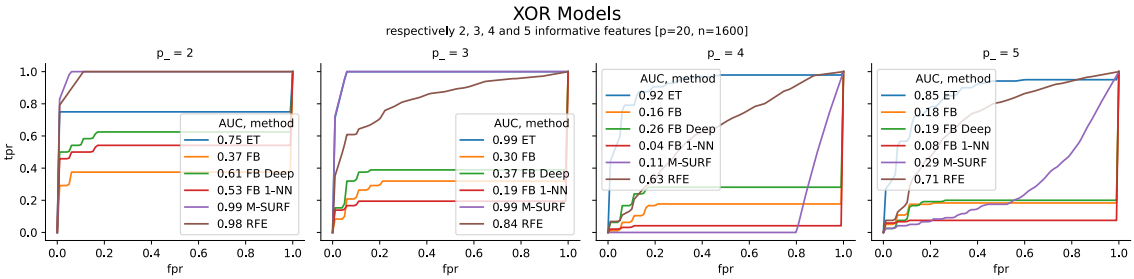


Figure 7: ROC curves for '*XOR Benchmarks*'. For the theme there exist 4 different models, with 4 different amount of informative features ranging from $p_{-} = 2$ to $p_{-} = 5$, exploring algorithm performance under higher-order interactions.

Raw data reveals that given the high feature dataset ($p = 10000$), all FeatBoost variants have their performance significantly impacted by both stopping early (subset sizes range from about 1 to 7) without having selected any correct features.

4.1.6 CPU timings

Figure 9 depicts CPU timings for the Total Number of Features datasets, as well as aggregates for all other datasets. It can be observed that FeatBoost and MultiSURF scale somewhat *linearly*, but RFE, like expected, *exponentially* - since RFE is exhaustive it requires significantly more time when more features are added, having to eliminate all features at all times. A curious result is the exceptionally long time MultiSURF took for computing the '135-bit multiplexer' dataset, taking the replicas 6.1 hours on average in comparison to just over 6 minutes for RFE. Since in the experiment all replicas ran on the same computation node, this might have been due to node malfunction. Another reason might be the high computational complexity that comes along with the 135-bit multiplexer - although there are 7 key features in this dataset, all 135 features are 'predictive'. Anyhow, the extra computational time MultiSURF had a positive impact in performance: for the 135-bit multiplexer

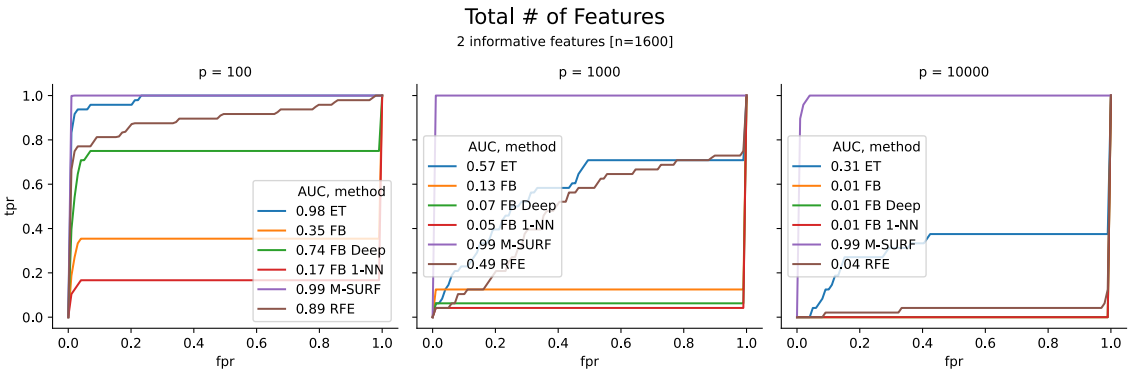


Figure 8: ROC curves for the '*Total Features*' dataset theme. Other dataset parameters (*Easy* model difficulty, $h = 0.4$) were kept constant, whilst varying the amount of dimensions.

MultiSURF scores highest with 0.99 AUC score, where other methods did significantly worse. In overall, ExtraTrees is by far the fastest - with all FeatBoost configurations as runner-ups. MultiSURF seems to take longer than FeatBoost on most configurations.

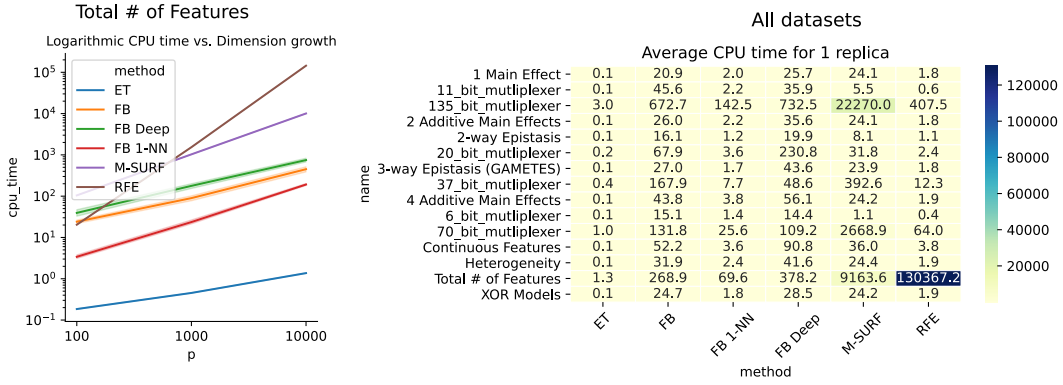


Figure 9: **Left**: CPU timings for Total Number of Features datasets, in a *logarithmic* scale. The steepness of the line could reveal the scalability of the algorithm. **Right** raw CPU timings for many datasets, aggregated over all configurations and replicas.

4.1.7 Concluding dataset themes

Albeit many dataset themes were discussed, more were tested. To stay concise, not all were included in the paper's main content. See Appendix A for more results.

4.2 Statistical integrity

Finally, after having empirically explored differences in feature selection algorithm performance, statistical tests can be performed to draw final conclusions. Like explained in Section 3.2.5, the *Wilcoxon signed ranks test* and the *Friedman test* are sensible tests to use given the contexts, recommended for comparing 2 estimators or more, respectively. Given that in this research, no preference exists to proof one algorithm's performance over the other, a general comparison is made using the Friedman test. When significance is shown the data will be analyzed post-hoc using the *Nemenyi post-hoc test*.

Let the **Null Hypothesis** be: *there is no significant difference in aggregated results between the 6 chosen feature selectors (groups) for 75 datasets (blocks)*. Given the various collected metrics,

three such hypotheses can be set for: $ROCAUC$, mAP, *stability*.

Metric aggregation is done in different ways given the various metrics: for $ROCAUC$ and mAP, one score exist for every feature selection method per replica. For any feature selector, all replica scores are *averaged*. For the *stability* metric, stability between replicas was measured for subset of various amounts of maximum length for each method, i.e. stability was measured for subset sizes with maximum length in the range $[1, 10]$. In this way, 10 stability scores per ranking method per dataset are obtained. The means of aggregating these scores, is to take the *maximum* value in the range, i.e. for every ranking method, the peak value as in Figure 3 is taken. This method of aggregation seems sensible, since including the stability scores of early iterations would steer the score downward, since (correct) features can be selected in different order. This measure of aggregating by taking the maximum values, does not, however, take *stopping* criteria into account. If all tested ranking methods would exhibit stopping criteria, instead of taking the maximum stability value, simply the stability of the selected subset could be taken, better resembling actual feature selection stability.

Figure 10 depicts **significance testing** results. A Friedman test showed significant differences for all metrics, as visible by p-values of < 0.05 in green colors. Because the null-hypothesis was therefore rejected for all metrics, a Nemenyi post-hoc test can be applied to find significance levels between groups. The Nemenyi post-hoc test showed that there exist multiple significant differences between groups. To allow for more meaningful interpretation, however, it can be useful to take the distributions of the metrics per feature selector into account.

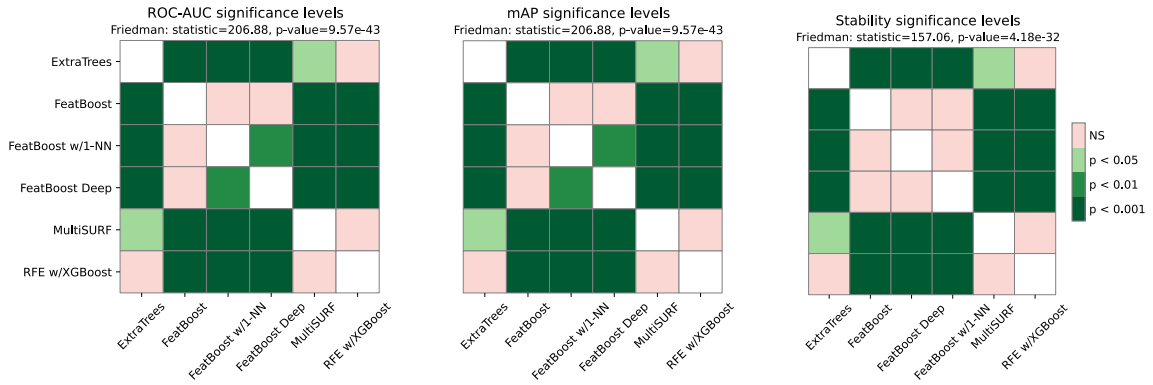


Figure 10: Significance plots for $ROCAUC$, mAP and *Stability* metrics, resulting in Friedman-test p-values of respectively 1.92×10^{-40}, 9.57×10^{-43} and 3.41×10^{-8}. The heatmap displays results for a Nemenyi post-hoc analysis: red shows no significance, darker green is more significant.

Figure 11 depicts **aggregated metrics** for every tested feature selection algorithm. The metrics have been aggregated over all datasets to obtain a distribution plot, for which median values were drawn using black bars. Combining the insights of this chart with the significance plots, a couple observations can be made. (1) There exist strong significance that ExtraTrees, RFE and MultiSURF score higher in terms of AUC and maP (subset quality) than all FeatBoost variants. There exist no significance when comparing RFE, but the MultiSURF vs ExtraTrees comparison is significant, indicating MultiSURF superiority. (2) Between FeatBoost configurations, there exist a significant difference between FeatBoost Deep and FeatBoost w/1-NN, indicating FeatBoost Deep superiority when combined with the average metric scorings. (3) In terms of stability, we again find similar significance levels, indicating better MultiSURF and RFE performance over FeatBoost variants, and better MultiSURF performance over ExtraTrees.

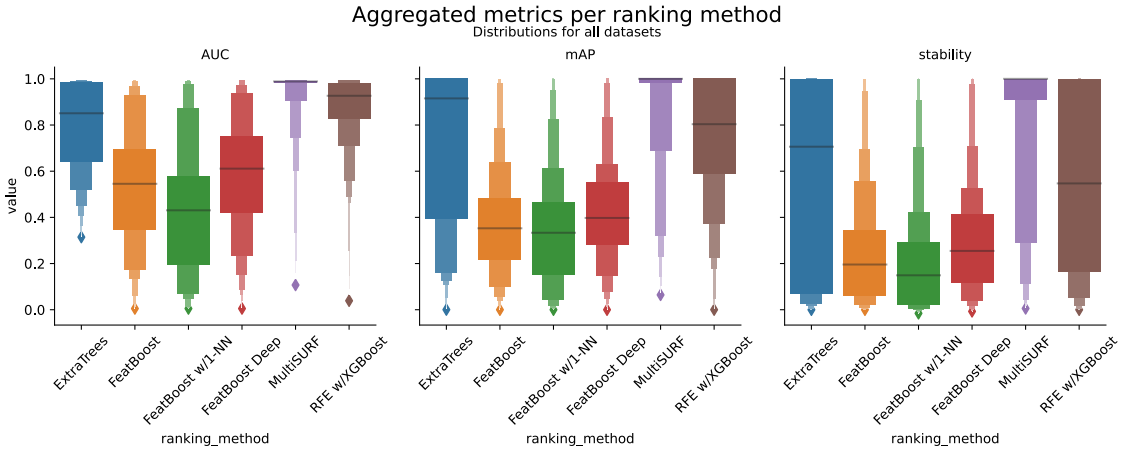


Figure 11: Various aggregated metrics over all datasets, given all ranking methods. The way of aggregation differs per metric, see Section 3.2.5.

5 Conclusions and Future Work

5.1 Conclusion

Traditionally, many authors relied solely on classification accuracy to measure feature selection performance. Using *a priori* knowledge about informative features, however, more information can be obtained about the quality of a feature subset, like commonly used machine learning metrics such as the ROC curve with its AUC and PR curves with a mAP score. Since feature selection evaluation is a multi-faceted problem, other metrics should also be taken into account besides measuring feature subset quality. Comprehensive evaluation should include analysis on feature selection method stability, complexity and scalability. To conclude an analysis, suitable statistical tests are the Wilcoxon signed ranks test or Friedman test with Nemenyi post-hoc test, when comparing two- or more feature selectors, respectively. To implement the newly proposed set of metrics into a pipeline, a framework called 'fseval' was proposed, to make for easy application of the evaluation pipeline. Lastly, a quantitative experiment has shown that using the proposed metrics allows for more thorough evaluation than when no a priori knowledge is used.

Feature selection is an ever more relevant problem in a world where data and machine learning pose a prevalent role. With many feature selection algorithms available, a comprehensive analysis is required to pick a suitable method given the context. But, when all relevant facets are highlighted and analyzed in a comprehensive feature selection evaluation pipeline, authors and users can be more deliberate in arguing any algorithm's superiority.

5.2 Future Work

Ideas for future authors include the following. (1) Given that, like discussed in the paper, the PR curves do not faithfully represent performance when an algorithm stops at various subset sizes over several algorithm runs, a slight adjustment to the PR curve might be done. An idea is to use some sort of *weighted* PR curve, which would take the amount of algorithm runs (replicas) into account in plotting the PR curve - e.g. the mean PR curve could be scaled at the precision levels w.r.t. the amount of replicas supporting the curve. In such a way, the graphical representation of performance given the PR curve might be more representative. (2) In the currently proposed set of metrics, every subsequently selected feature has the same 'weight', i.e. the recall and precision scores scale linearly. It is often the case, however, that a prediction task only succeeds when all informative features are selected, not a part of them. For this reason, algorithms that selected only a part of the informative feature set, but not all, might be overly optimistically rewarded in recall or precision points. An idea would be, to reward algorithms in a different than linear (exponential, quadratic, for example)

fashion for selecting more correct features, e.g. given 3 informative features true positive scorings could be $\{0, 0.11, 0.33, 1.0\}$, exponentially rewarding selecting more correct features. More research on such metric would be in place, e.g. investigating varying levels of f_{beta} scores.

Limitations of this paper are several - which of course exist, although this paper tried to construct a novel feature selection evaluation pipeline as comprehensive as possible. (1) First of all, the proposed evaluation pipeline could have been more complete, e.g. by investigating how algorithm simplicity can be quantified. (2) Secondly, the experiment could have been more comprehensive. Even though it was recommended in the proposed evaluation pipeline to analyse algorithm complexity by means of theoretical analysis, the scope of the experiment did not allow for such thorough analysis. (3) Third, a more consistent analysis can probably be drawn by including only algorithms of the same category, i.e. only algorithms that exhibit stopping criteria, or only algorithms that produce a ranking. Because in the quantitative part of the paper a mixed bag was used, it was harder to isolate the strength of some feature selector's stopping criteria (e.g. FeatBoost and ExtraTrees). (4) Lastly, the experiment could have included more datasets with higher dimensionality, to provide more insight on the scalability and chosen subset qualities of the algorithms. Only a few of the used datasets were of ($p \gg n$) type, where the amount of features greatly outnumbers the amount of samples.

References

[1] Q. Al-Tashi, H. M. Rais, S. J. Abdulkadir, S. Mirjalili, and H. Alhussian. A review of grey wolf optimizer-based feature selection methods for classification. In *Evolutionary Machine Learning Techniques*, pages 273–286. Springer, 2020.

[2] S. Alelyani. *On feature selection stability: A data perspective*. Citeseer, 2013.

[3] V. Bolón-Canedo and A. Alonso-Betanzos. Ensembles for feature selection: A review and future trends. *Information Fusion*, 52:1–12, 2019.

[4] V. Bolón-Canedo, N. Sánchez-Maroño, and A. Alonso-Betanzos. A review of feature selection methods on synthetic data. *Knowledge and information systems*, 34(3):483–519, 2013.

[5] K. Boyd, K. H. Eng, and C. D. Page. Area under the precision-recall curve: point estimates and confidence intervals. In *Joint European conference on machine learning and knowledge discovery in databases*, pages 451–466. Springer, 2013.

[6] Z. Cai, R. Li, and L. Zhu. Online sufficient dimension reduction through sliced inverse regression. *Journal of Machine Learning Research*, 21(10):1–25, 2020.

[7] G. Chandrashekar and F. Sahin. A survey on feature selection methods. *Computers & Electrical Engineering*, 40(1):16–28, 2014.

[8] P. Cunningham. Dimension reduction, technical report ucd-csi-2007-7. *University College Dublin*, 2007.

[9] S. Das. Filters, wrappers and a boosting-based hybrid for feature selection. In *Icml*, volume 1, pages 74–81, 2001.

[10] J. Davis and M. Goadrich. The relationship between precision-recall and roc curves. In *Proceedings of the 23rd international conference on Machine learning*, pages 233–240, 2006.

[11] J. Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine learning research*, 7(Jan):1–30, 2006.

[12] K. Dunne, P. Cunningham, and F. Azuaje. Solutions to instability problems with sequential wrapper-based approaches to feature selection. *Journal of Machine Learning Research*, pages 1–22, 2002.

[13] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman. The pascal visual object classes (voc) challenge. *International journal of computer vision*, 88(2):303–338, 2010.

[14] P. Geurts, D. Ernst, and L. Wehenkel. Extremely randomized trees. *Machine learning*, 63(1):3–42, 2006.

[15] I. Guyon and A. Elisseeff. An introduction to variable and feature selection. *Journal of machine learning research*, 3(Mar):1157–1182, 2003.

[16] I. Guyon, J. Weston, S. Barnhill, and V. Vapnik. Gene selection for cancer classification using support vector machines. *Machine learning*, 46(1-3):389–422, 2002.

[17] H.-H. Hsu, C.-W. Hsieh, and M.-D. Lu. Hybrid feature selection by combining filters and wrappers. *Expert Systems with Applications*, 38(7):8144–8150, 2011.

[18] A. Kalousis, J. Prados, and M. Hilario. Stability of feature selection algorithms: a study on high-dimensional spaces. *Knowledge and information systems*, 12(1):95–116, 2007.

[19] R. A. Khurma, I. Aljarah, A. Sharieh, and S. Mirjalili. Evolopy-fs: An open-source nature-inspired optimization framework in python for feature selection. In *Evolutionary Machine Learning Techniques*, pages 131–173. Springer, 2020.

[20] K. Kira, L. A. Rendell, et al. The feature selection problem: Traditional methods and a new algorithm. In *Aaai*, volume 2, pages 129–134, 1992.

[21] D. Koller and M. Sahami. Toward optimal feature selection. Technical report, Stanford InfoLab, 1996.

[22] P. Křížek, J. Kittler, and V. Hlaváč. Improving stability of feature selection methods. In *International Conference on Computer Analysis of Images and Patterns*, pages 929–936. Springer, 2007.

[23] L. I. Kuncheva. A stability index for feature selection. In *Artificial intelligence and applications*, pages 421–427, 2007.

[24] J. Li, K. Cheng, S. Wang, F. Morstatter, R. P. Trevino, J. Tang, and H. Liu. Feature selection: A data perspective. *ACM Computing Surveys (CSUR)*, 50(6):1–45, 2017.

[25] X. Li, R. Li, Z. Xia, and C. Xu. Distributed feature screening via componentwise debiasing. *Journal of Machine Learning Research*, 21(24):1–32, 2020.

[26] M. Mafarja, A. A. Heidari, H. Faris, S. Mirjalili, and I. Aljarah. Dragonfly algorithm: theory, literature review, and application in feature selection. In *Nature-Inspired Optimizers*, pages 47–67. Springer, 2020.

[27] P. Mohana Chelvan and K. Perumal. A survey on feature selection stability measures. *International Journal of Computer and Information Technology*, 5(1):98–103, 2016.

[28] P. Nemenyi. Distribution-free multiple comparisons (doctoral dissertation, princeton university, 1963). *Dissertation Abstracts International*, 25(2):1233, 1963.

[29] S. Nogueira, K. Sechidis, and G. Brown. On the stability of feature selection algorithms. *The Journal of Machine Learning Research*, 18(1):6345–6398, 2017.

[30] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al. Scikit-learn: Machine learning in python. *the Journal of machine Learning research*, 12:2825–2830, 2011.

[31] G. Salton and M. J. McGill. *Introduction to Modern Information Retrieval*. McGraw-Hill, Inc., USA, 1986.

[32] S. Solorio-Fernández, J. A. Carrasco-Ochoa, and J. F. Martínez-Trinidad. A review of unsupervised feature selection methods. *Artificial Intelligence Review*, 53(2):907–948, 2020.

[33] C. Y. Tang, E. X. Fang, and Y. Dong. High-dimensional interactions detection with sparse principal hessian matrix. *Journal of Machine Learning Research*, 21(19):1–25, 2020.

[34] J. Tang, S. Alelyani, and H. Liu. Feature selection for classification: A review. *Data classification: Algorithms and applications*, page 37, 2014.

[35] B. Thomee, D. A. Shamma, G. Friedland, B. Elizalde, K. Ni, D. Poland, D. Borth, and L.-J. Li. Yfcc100m: The new data in multimedia research. *Communications of the ACM*, 59(2):64–73, 2016.

[36] R. J. Urbanowicz, M. Meeker, W. La Cava, R. S. Olson, and J. H. Moore. Relief-based feature selection: Introduction and review. *Journal of biomedical informatics*, 85:189–203, 2018.

[37] R. J. Urbanowicz, R. S. Olson, P. Schmitt, M. Meeker, and J. H. Moore. Benchmarking relief-based feature selection methods for bioinformatics data mining. *Journal of biomedical informatics*, 85:168–188, 2018.

[38] M. Zhu. Recall, precision and average precision. *Department of Statistics and Actuarial Science, University of Waterloo, Waterloo*, 2:30, 2004.

Appendices

A Additional results

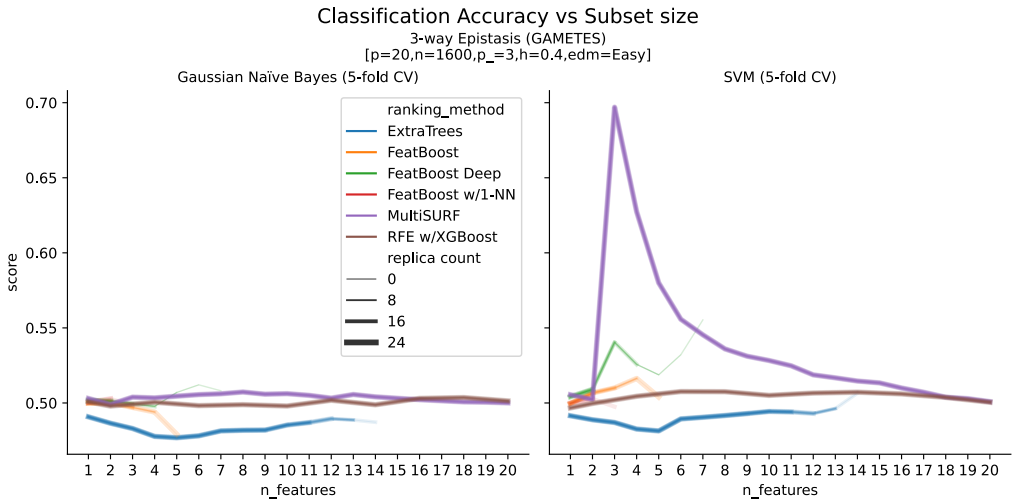


Figure 12: Two more results accompanying the graphs in Figure 4: Classification Accuracy versus Feature Subset size, for the '3-way Pure Epistasis' dataset.

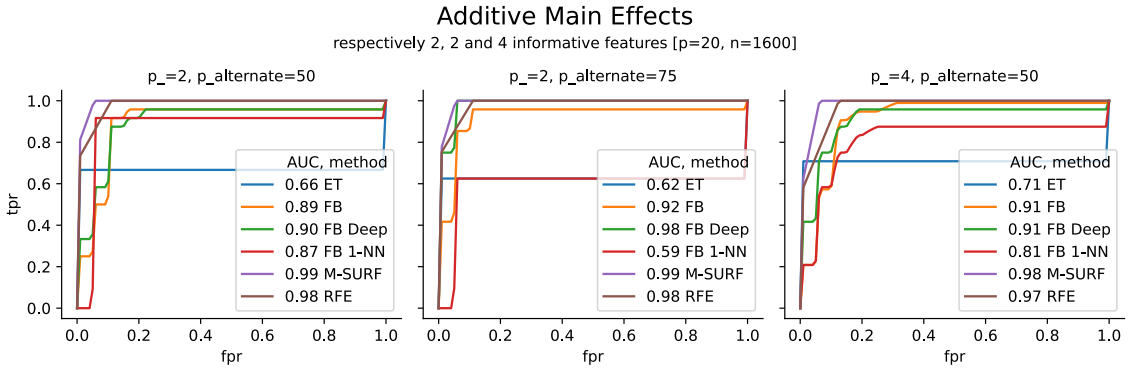


Figure 13: ROC curves for '*Additive Main Effects*' dataset theme.

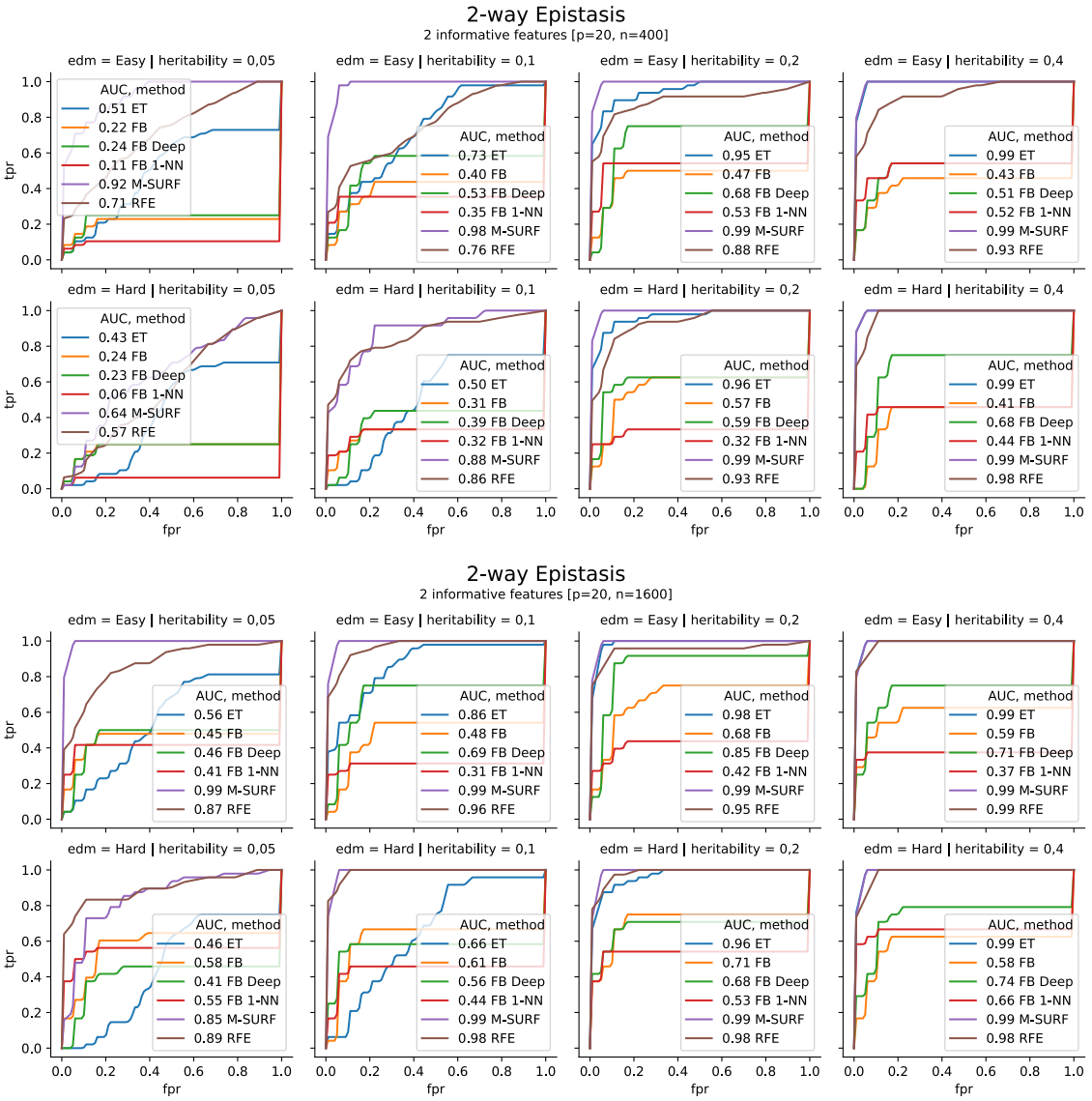


Figure 14: ROC curves for '*2-way Epistasis*' dataset theme, for larger sample sizes, accompanying Figure 6.

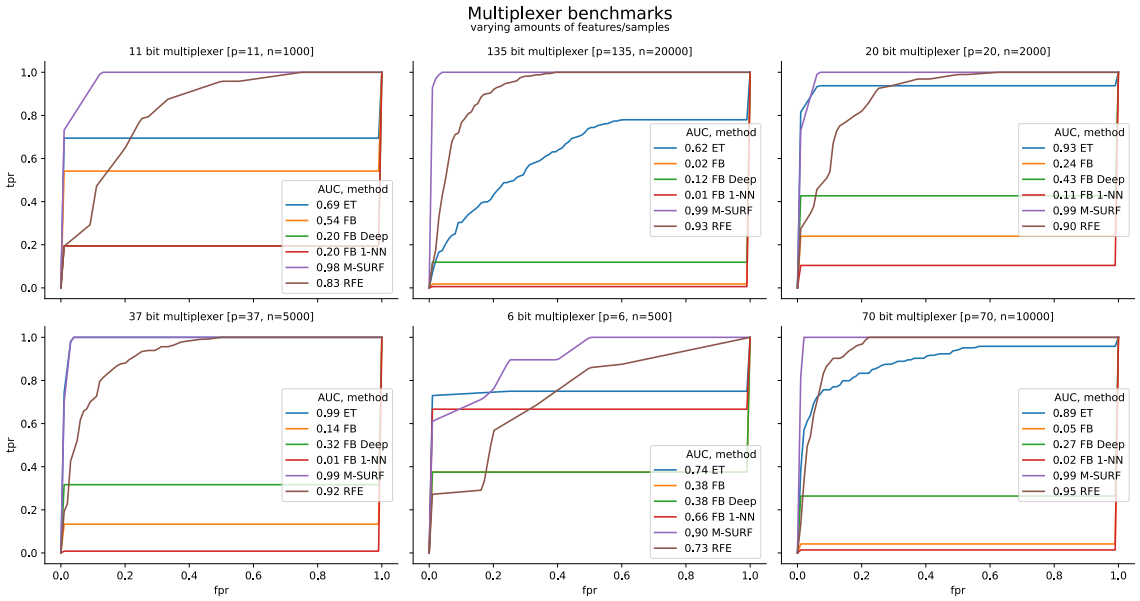


Figure 15: ROC curves for the '*Multiplexer*' dataset theme.

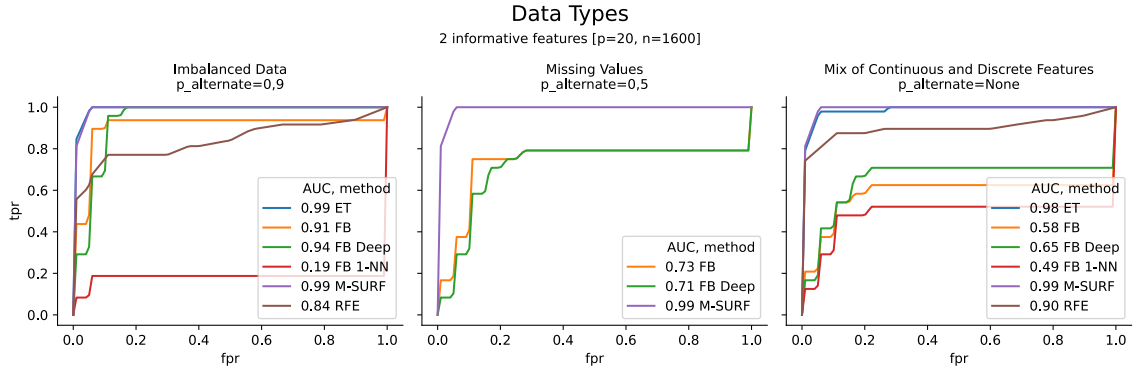


Figure 16: ROC curves for the '*Data Types*' dataset theme.

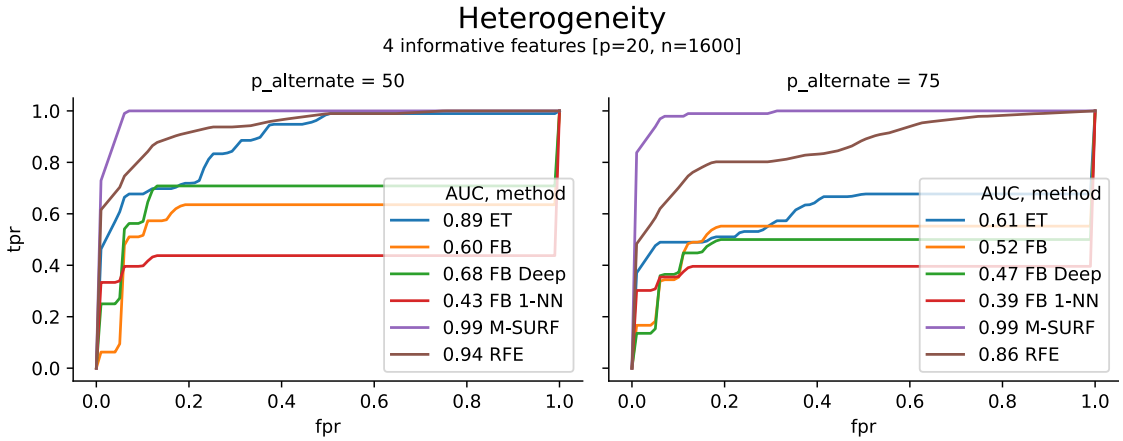


Figure 17: ROC curves for '*Genetic Heterogeneity*' dataset theme. 2 configurations were tested.